LES 5 – AUTOMATISERING

Alimkhan Salamov

Janiek Colpaert

hogeschool vives

Inhoud

Externe modules installeren

- Pip
- Virtual environment

Netmiko

Github

Labo: Switch configuratie met Python

EXTERNE MODULES INSTALLEREN

Externe modules

Een van de krachtigste eigenschappen van Python is de enorme community die duizenden pakketten heeft ontwikkeld om bijna elk probleem op te lossen.

Deze pakketten voegen functionaliteit toe aan je projecten, van webscraping tot machine learning. Om deze pakketten te beheren, gebruik je **pip.** Python's ingebouwde package manager.

pip (afkorting voor "Pip Installs Packages") is de standaard tool in Python om externe pakketten te installeren en beheren. Het stelt je in staat om toegang te krijgen tot een enorme bibliotheek van open-source tools en frameworks die beschikbaar zijn via de **Python Package Index (PyPI).**

Installeren van een pakket

Om een pakket te installeren, gebruik je de opdracht:

```
pip install <pakketnaam>
```

Voorbeeld: Het pakket `requests` installeren. Het pakket requests wordt vaak gebruikt om gegevens van websites op te halen via HTTP.

```
pip install requests
```

<u>Wat gebeurt er?</u>

pip downloadt het pakket van PyPI.

Het pakket wordt geïnstalleerd en is direct beschikbaar in je Python-project.

Gebruik extern pakket

Laten we requests gebruiken om een eenvoudige HTTP-aanvraag te doen.

```
import requests

# Maak een GET-verzoek naar een API
response = requests.get("https://api.chucknorris.io/jokes/random")

# Controleer de statuscode van het antwoord
print(f"Statuscode: {response.status_code}")
# Toont: 200
print(response.text)
```

Wat gebeurt hier?

- requests.get() maakt een HTTP-aanvraag naar de opgegeven URL.
- response.status_code geeft de HTTP-statuscode terug:
- 200: Succes.
- 404: Niet gevonden.
- 500: Serverfout.

Virtuele omgevingen

Het is een goede gewoonte om een virtuele omgeving te gebruiken voor elk project. Hiermee kun je pakketversies isoleren en conflicten vermijden.

Hoe maak je een virtuele omgeving?

1. Maak een nieuwe virtuele omgeving:
   ```
   python -m venv .venv
   ```
2. Activeer de virtuele omgeving:
   - **Windows:** .\.venv\Scripts\activate
   - **Mac/Linux:** source .venv/bin/activate
3. Installeer pakketten binnen de omgeving:
   ```
   pip install requests
   ```
4. Deactiveer de virtuele omgeving:
   ```
   deactivate
   ```

CONNECTEREN MET EEN DEVICE

Netmiko

Netmiko is een open-source Python library dat SSH management vereenvoudigt met netwerkapparaten. De library is gebaseerd op de Paramiko SSH library.

Het **doel** is om:

- Verbinding te kunnen maken met een netwerk apparat
- Het eenvoudig te maken om commando's op die apparaten uit te voeren en data uit te lezen van show commands
- Het eenvoudig te maken om configuratie-commando's uit te voeren
- En dit met een brede aantal merken van apparaten.

Naast SSH kan je ook verbinding maken via:

- Secure Copy
- Telnet
- Serial connections (console kabel)

De ondersteuning hiervan is wel minder dan via SSH

Connectie maken via console kabel

Er bestaan ook andere libraries om verbinding te maken met een netwerkkabel. Een voorbeeld hiervan is **PySerial**. Dit lukt echter ook via Netmiko, al is de ondersteuning hier wel wat minder.

Maak eerst een virtual environment aan. Installeer daarna netmiko met volgende commando:

```
pip install netmiko
```

Maak een bestand aan `console_connect_cable.py`

Maak het mogelijk om de netmiko-library te gebruiken door volgende code bovenaan toe te voegen aan het bestand:

```
from netmiko import ConnectHandler
```

Switch aansluiten

Sluit de switch aan en voer volgende code uit, wat zie je?

```
from netmiko import ConnectHandler

platform = 'cisco_ios_serial'
port = 'COM3'
baudrate = 9600

device_object = ConnectHandler(device_type=platform, serial_settings = {
    "port": port,
    "baudrate": baudrate
})

print(device_object.send_command("show version"))
```

Output gebruiken (1)

De console output is vaak moeilijk te lezen.

Een manier die je al gezien hebt is om dit via een commando weg te schrijven naar een bestand.

Hoe kon je dit doen?

```
python console_connect_cable.py > show_version.txt
```

Of kan je de tekst via code wegschrijven?

Output gebruiken (2)

We kunnen hier echter nog niet mee aan de slag.

Je kan echter de data wel gestructureerd meekregen. Daarvoor dien je een extra parameter mee te geven met de send_command functie:

```
print(device_object.send_command("show version", use_textfsm=True))
```

Kijk naar de output welk datatype krijg je terug?

Hoe zou je de hostnaam er kunnen uithalen?

WERKEN MET GIT

Wat gaan we leren?

- Wat is Git? Wat is GitHub?
- Git installeren en configureren
- Repo aanmaken, commits maken, pushen
- .gitignore en README.md
- Git clone
- **Hands-on oefening**

Wat is Git?

- **Versiebeheersysteem** (version control)
- Draait **lokaal** op je computer
- Houdt de **volledige geschiedenis** bij van je bestanden
- Je kan altijd **terug naar een vorige versie**

= ONEdrive voor code

Wat is GitHub?

- **Online platform** om Git-repositories te hosten
- Samenwerken met anderen
- Backup in de cloud
- Extra functies: issues, pull requests, …

Git vs GitHub

- **Git** = de tool
- **GitHub** = de dienst

Git	GitHub
Software op je PC	Website / online dienst / platform
Werkt lokaal	Werkt in de cloud
Houdt geschiedenis bij	Host je repo online

Waarom is dit belangrijk?

- **Geschiedenis** — elke wijziging wordt bijgehouden
- **Samenwerking** — meerdere mensen, één project
- **Backup** — code staat veilig op GitHub
- **Professioneel** — Git is de standaard in de IT

Belang voor netwerkbeheerders:

- Bijhouden van configuratiebestanden en hun geschiedenis
- Mogelijkheid om bij aanpassing van de configuratie dit automatisch toe te passen op een omgeving (CI/CD – configuration as code)

Belangrijke concepten

Concept	Betekenis
Repository (repo)	Opslagplaats voor bestanden + geschiedenis
Commit	Snapshot van je wijzigingen
Branch	Parallelle versie van je project
Merge	Branches samenvoegen
Clone	Kopie van remote repo → lokaal
Remote	Repo op server (GitHub)
Lokaal	Repo op je eigen PC

Git installeren (zie handleiding)

- Download van **git-scm.com**
- Volg de installatie (handleiding beschikbaar)
- Controleer in terminal:

```
git --version
```

Git configureren

- Stel je naam en e-mail in (**gebruik je VIVES-account**):
- Waarom? → elke commit bevat deze info

```
git config --global user.name "Voornaam Achternaam"
git config --global user.email "studentnummer@student.vives.be"
```

Configuratie controleren

```
git config --global --list
```

Je ziet:

user.name=Voornaam Achternaam

user.email=studentnummer@student.vives.be

Wat is een repository?

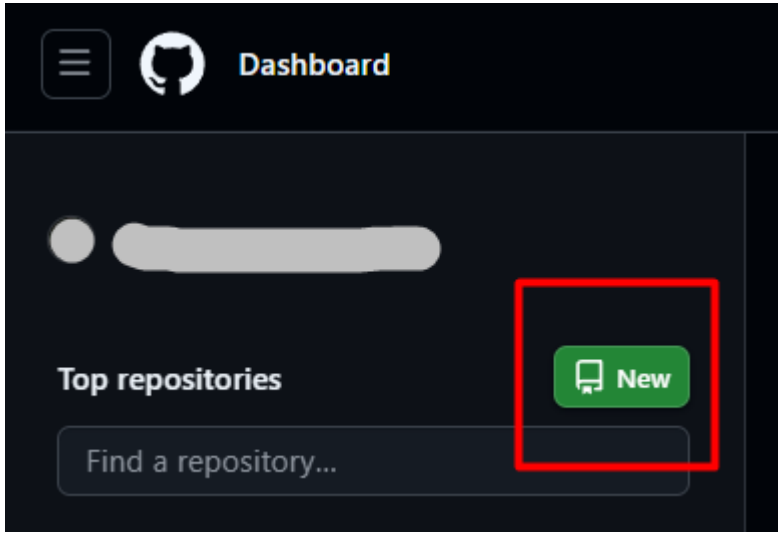
- Een map met projectbestanden
- Git houdt alle wijzigingen bij
- **Lokaal**: op je PC (met .git/ folder)
- **Remote**: op GitHub (online)

De .git/ folder

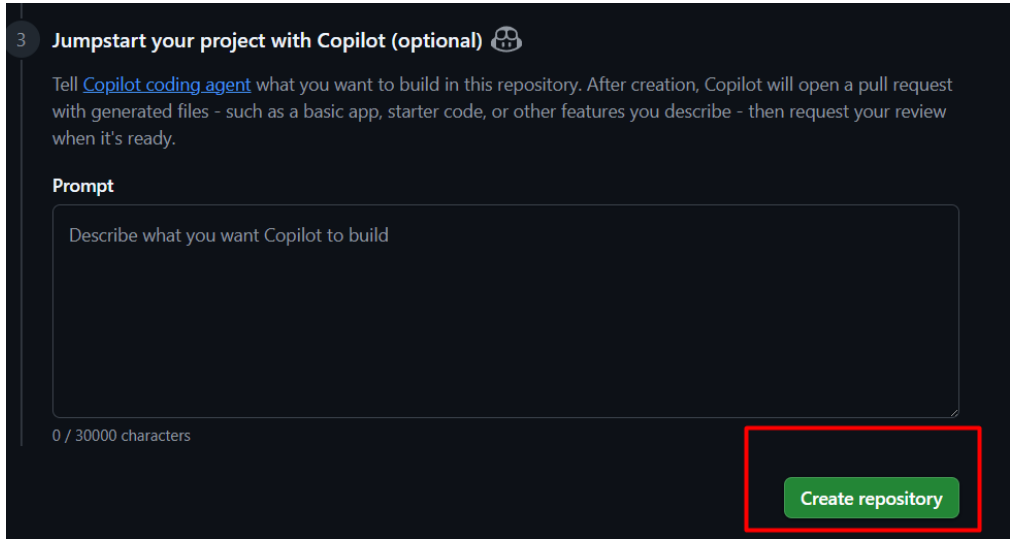
- Verborgen map in je project
- Bevat:
  - Volledige **geschiedenis** (alle commits)
  - **Configuratie** (o.a. welke remote)
  - Info over **branches**
- **Nooit verwijderen!**

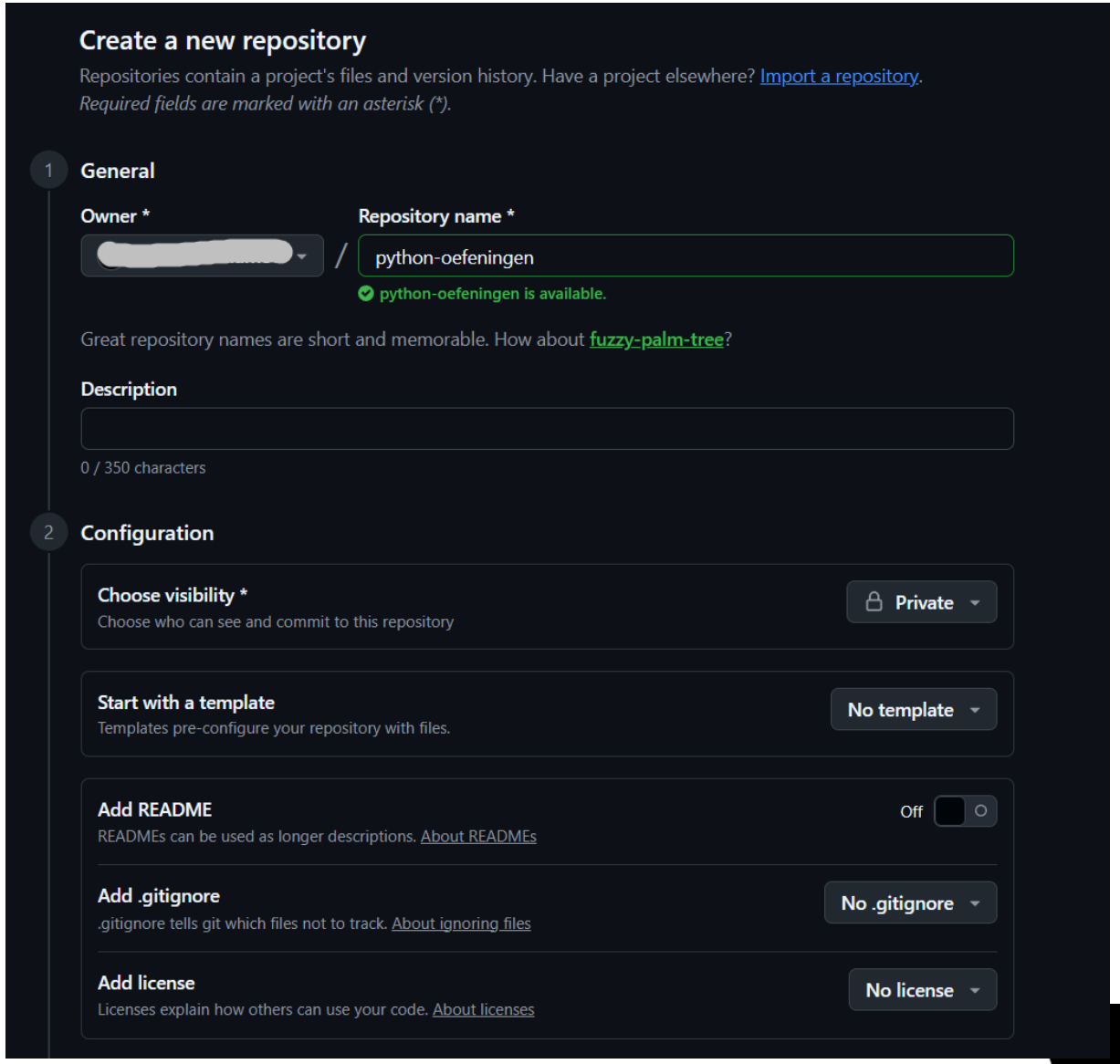
Repo aanmaken — Op GitHub

- Ga naar **github.com** → log in
- Klik op **“New"**
- Vul in:
  - Naam: python-oefeningen
  - Laat opties **uitgevinkt**
- Klik **"Create repository"**



- Ga naar **github.com** → log in
- Klik op **“New"**
- Vul in:
  - Naam: python-oefeningen
  - Laat opties **uitgevinkt**
- Klik **"Create repository"**





Repo aanmaken — Lokaal

- Open een terminal (of in VS Code):

```
mkdir python-oefeningen
cd python-oefeningen
git init
```

- → Je map is nu een Git-repo (.git/ verschijnt)

Remote koppelen

- Koppel je lokale repo aan GitHub:

```
git remote add origin https://github.com/JOUW-USERNAME/python-oefeningen.git
```

- Controleren:

```
git remote -v
```

De basis-workflow

Werkmap —— git add ——▶ Staging —— git commit ——▶ Lokale repo —— git push ——▶ GitHub

4 stappen die je **steeds herhaalt**.

git status

- Bekijk wat er veranderd is:

```
git status
```

- Toont:
  - Nieuwe bestanden
  - Gewijzigde bestanden
  - Bestanden klaar om te committen

git add

Zet bestanden **klaar** om te committen:

```
git add bestandsnaam.py      # één specifiek bestand
git add .                    # ALLES in de huidige map
```

git commit

Maak een **snapshot** met een boodschap:

```
git commit -m "beschrijving van wat je veranderd hebt"
```

Tip: schrijf **duidelijke** boodschappen!

git push

Stuur je commits naar GitHub:

```
git push
```

Eerste keer:

```
git push -u origin main
```

Daarna volstaat

```
git push
```

git pull

- Haalt de **nieuwste commits** van GitHub op
- Combineert die met jouw lokale branch

```
git pull
```

Gebruik dit:

- Als je op meerdere pc's aan dezelfde repo werkt
- Als anderen eerst veranderingen hebben gepusht

.gitignore

Niet alle bestanden horen in Git. Maak een .gitignore:

```
.venv/
__pycache__/
*.pyc
.env
```

Patroon	Wat?
.venv/	Virtuele omgeving
__pycache__/	Python cache
***.pyc**	Gecompileerde Python
.env	Wachtwoorden / keys

Overzicht van handige commando's

Commando	Wat?
git init	Maak een repo
git clone <url>	Kopieer een repo
git status	Bekijk status
git add .	Alles klaarzetten
git commit -m "msg"	Snapshot maken
git push	Naar GitHub sturen
git pull	Van GitHub ophalen
git log --oneline	Geschiedenis bekijken
git remote -v	Remote tonen

LABO

Vul labo aan

Je kan het labo terugvinden op toledo.

Aanvaard daar de classroom link. Aanvaard de uitnodiging die je krijgt in je mailbox.

Je krijgt tijd tot het einde van de les. Indien je klaar bent, verwittig je de docent. Indien ok, kan je de les verlaten.